



UNIVERSIDAD DE MONTERREY

Práctica 5

Microprocesadores
M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo GarcíaMat: 612348

Nombre: Ricardo Garza BenítezMat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 9 de Junio del 2026

Introducción

El manejo de displays de siete segmentos mediante multiplexación y la lectura de entradas digitales son dos de las habilidades más fundamentales en el desarrollo de sistemas embebidos con microcontroladores. En esta práctica se trabaja con el PIC16F887 a una frecuencia de oscilación de 16 MHz, empleando el compilador XC8 y el entorno MPLAB X. El objetivo central es implementar sistemas de conteo visual sobre distintas configuraciones de displays de siete segmentos, combinando el control de salidas digitales con la detección de eventos en botones físicos.

Las tres actividades de la práctica abordan el problema desde ángulos complementarios. La Actividad A explora la multiplexación de cuatro dígitos para desplegar un contador ascendente/descendente de cuatro cifras (0–9999) cuya dirección se invierte al presionar un botón. La Actividad B introduce el concepto de interrupción por hardware, deteniendo el conteo en un display de un dígito para parpadear un LED y luego reanudando desde el valor en el que se interrumpió. Finalmente, la Práctica 1 trabaja una configuración de display de siete segmentos de cuatro dígitos donde se detecta el rebote mecánico del botón y se incorpora un retardo de debounce por software para garantizar lecturas estables.

Un elemento crítico presente en las tres actividades es la configuración correcta de los registros ANSEL y ANSELH del PIC16F887. Dado que varios pines del puerto B comparten función analógica, es indispensable establecerlos como entradas digitales antes de intentar leer el estado de cualquier botón conectado a ese puerto. Sin este paso, el microcontrolador interpretará las señales digitales como niveles analógicos y el sistema no responderá a las pulsaciones, lo cual fue precisamente uno de los primeros problemas encontrados durante el desarrollo de las actividades.

Marco Teórico

Displays de siete segmentos y multiplexación

Un display de siete segmentos es un dispositivo de visualización formado por siete LEDs individuales (etiquetados de 'a' a 'g') más un segmento de punto decimal. Al activar combinaciones específicas de estos segmentos se representan los dígitos del 0 al 9, así como varios caracteres alfanuméricos. Existen dos configuraciones comerciales: ánodo común, donde todos los ánodos comparten un nodo positivo y el segmento se activa llevando su cátodo a nivel bajo; y cátodo común, donde todos los cátodos comparten el nodo de tierra y el segmento se activa elevando su ánodo a nivel alto. En esta práctica se emplean displays de cátodo común, por lo que los patrones de activación son activos en alto y la selección de dígito se realiza llevando la línea de habilitación correspondiente a nivel bajo [1].

Cuando se requiere mostrar más de un dígito de forma simultánea con un número reducido de pines de salida, se recurre a la técnica de multiplexación temporal. Esta técnica consiste en activar un solo display a la vez durante un intervalo de tiempo muy corto (típicamente entre 1 y 5 ms), mientras los demás permanecen apagados. El ciclo se repite con suficiente rapidez —mayor a 50 Hz— para que la persistencia de la visión

humana perciba todos los dígitos como encendidos al mismo tiempo. En la práctica, el Puerto D del PIC16F887 entrega el patrón de segmentos y el Puerto C selecciona cuál de los cuatro dígitos está habilitado en cada instante del ciclo [1].

Registro ANSEL y ANSELH — Configuración de entradas digitales

El PIC16F887 dispone de dos registros de selección analógico/digital: ANSEL, asociado al Puerto A y parte del Puerto E, y ANSELH, asociado a los pines AN8–AN13 del Puerto B. Al encender el microcontrolador, todos estos bits quedan en '1' por defecto, lo que configura los pines compartidos como entradas analógicas. Si se intenta leer un nivel lógico en un pin cuyo bit ANSEL o ANSELH es '1', el módulo ADC interfiere con la lectura digital y el pin retorna valores indefinidos. Por ello, en cualquier aplicación que use el Puerto B como entrada digital —como ocurre en esta práctica con los botones conectados a RB0— es obligatorio escribir cero en el bit correspondiente de ANSELH antes de configurar el TRIS [2].

Interrupciones externas — INT (RB0/INT)

Una interrupción es un mecanismo por el cual un evento externo o interno puede suspender la ejecución del programa principal para atender una rutina de servicio de interrupción (ISR), retomando después el flujo original exactamente donde se detuvo. El PIC16F887 cuenta con una interrupción externa dedicada en el pin RB0/AN12/INT, configurable para dispararse en flanco de subida o de bajada mediante el bit INTEDG del registro OPTION_REG. Para habilitarla se deben activar el bit INTE del registro INTCON y el bit de habilitación global GIE. Dentro de la ISR es responsabilidad del programador limpiar la bandera INTF antes de retornar, ya que de lo contrario la interrupción se volvería a disparar de manera inmediata [2].

Debounce por software

El rebote mecánico (bounce) es un fenómeno inherente a los contactos físicos de cualquier pulsador o interruptor. Cuando se presiona un botón, las láminas metálicas del contacto rebotan brevemente antes de estabilizarse, generando múltiples transiciones lógicas en un intervalo de pocos milisegundos. Si el firmware lee el estado del botón sin ningún tipo de filtrado, puede interpretar una sola pulsación como varios eventos distintos. La solución por software más sencilla consiste en introducir un retardo después de detectar el primer flanco activo —habitualmente entre 5 y 20 ms— y luego volver a verificar el estado del pin para confirmar que la pulsación es real. En el código de esta práctica se utiliza `__delay_ms(5)` con el cristal a 16 MHz, valor suficiente para eliminar el rebote sin generar una latencia perceptible para el usuario [1].

Pull-ups internos del Puerto B

Para que un pin de entrada presente un nivel lógico alto definido cuando el botón está abierto —sin necesidad de resistores externos—, el PIC16F887 incorpora resistencias de pull-up internas en el Puerto B. Estas se habilitan de manera global desactivando el bit `nRBPU` del registro `OPTION_REG` (escribir '0'), y de manera individual activando el bit correspondiente en el registro `WPUB`. Esta combinación evita que la entrada quede flotante y garantiza lecturas estables en estado de reposo. En las actividades de esta

práctica se utiliza el pull-up interno del pin RB0 tanto en la detección por flanco de software como en la interrupción externa [2].

Simulación

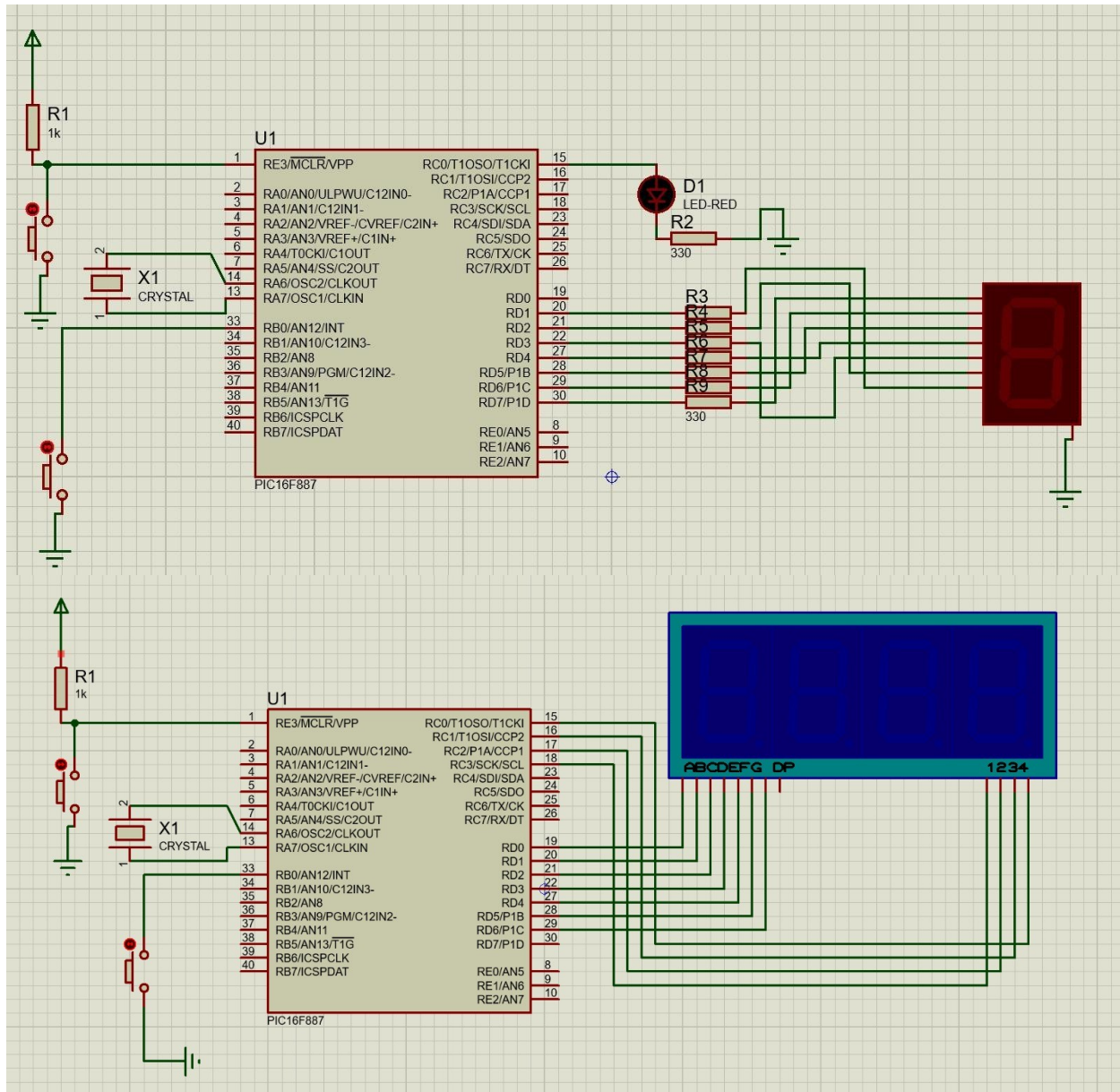
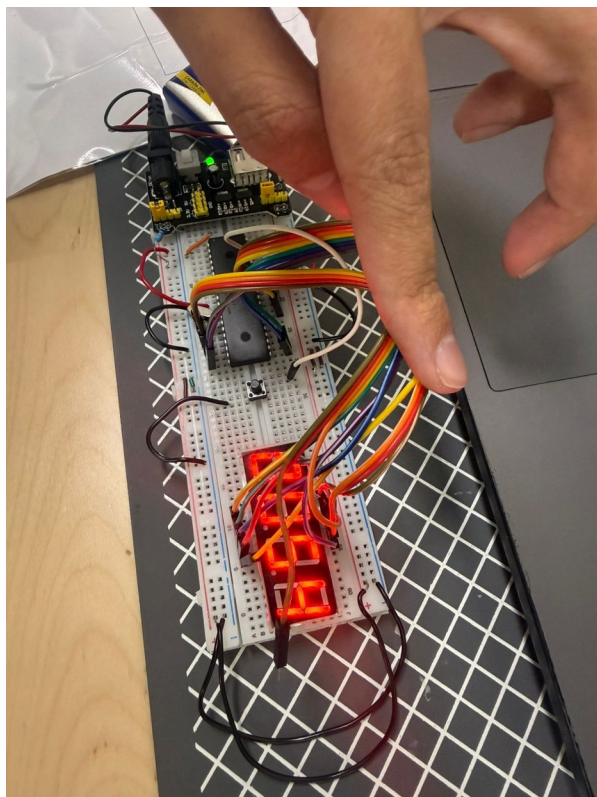


Foto del circuito en Físico



Resultados

Actividad A — Contador bidireccional 0–9999 con cuatro displays

En la Actividad A se implementó un contador ascendente/descendente con rango de 0 a 9999 sobre cuatro displays de siete segmentos multiplexados. El Puerto D entrega el patrón de segmentos extraído del arreglo `patron[]` y el Puerto C habilita secuencialmente cada dígito a través de un ciclo de multiplexación de 1 ms por dígito, resultando en un refresco visual de aproximadamente 25 Hz por cifra. Al presionar el botón conectado a RB0, el programa detecta el flanco descendente comparando el estado actual con el estado anterior (variable `ant`) y conmuta la variable `dir` entre +1 y -1, lo que invierte inmediatamente la dirección de conteo sin perder el valor acumulado. Al llegar a 9999 el contador regresa a 0 en modo ascendente, y al llegar a 0 salta a 9999 en modo descendente, logrando así una cuenta circular continua.

Durante las pruebas se observó que la multiplexación a 1 ms por dígito produce una imagen visualmente estable, sin parpadeo perceptible. La principal dificultad encontrada al armar la simulación y el circuito físico fue la identificación correcta del pinout del display de siete segmentos: se confundió inicialmente la configuración de ánodo común con la de cátodo común, lo que provocó que los segmentos no encendieran. Una vez corregida la orientación del display y verificados los patrones hexadecimales del arreglo, todos los dígitos se mostraron correctamente. La detección de flanco del botón funcionó de manera confiable y el cambio de dirección fue inmediato y preciso en cada pulsación.

Actividad B — Interrupción externa con pausa y parpadeo de LED

La Actividad B agrega al contador de un dígito en display de siete segmentos la gestión de una interrupción externa por hardware en RB0/INT. Al presionar el botón se dispara la ISR, la cual pausa el conteo, hace parpadear el LED conectado a RC2 durante un número determinado de ciclos y luego retorna al programa principal manteniendo el valor del contador en el punto exacto donde fue interrumpido. La bandera INTF se limpia al inicio de la ISR para evitar disparos repetidos. El estado del display se preserva porque la variable del contador es global y la ISR únicamente la congela durante el parpadeo sin modificar su valor.

Los resultados obtenidos mostraron que la interrupción responde de forma instantánea al flanco del botón sin necesidad de encuestas continuas en el loop principal, liberando al procesador de revisar el estado del pin en cada iteración. El LED parpadeó con la cadencia esperada y, al terminar la rutina de interrupción, el contador reanudó exactamente desde el dígito en que se encontraba. Se confirmó también que sin la configuración de ANSELH = 0 la interrupción no se disparaba, validando la importancia crítica de esta instrucción de inicialización cuando se usa RB0 como entrada digital.

Práctica 1 — Contador con rebote en display de cuatro dígitos

La Práctica 1 retoma la estructura de conteo sobre cuatro displays multiplexados e introduce el problema del rebote mecánico de manera deliberada para luego resolverlo por software. Sin el retardo de debounce, al presionar rápidamente el botón el microcontrolador detectaba múltiples flancos en un mismo evento físico, incrementando o decrementando el contador más de una vez por pulsación. Al añadir `__delay_ms(5)` después del primer flanco detectado, el comportamiento se estabilizó y cada pulsación produjo exactamente un cambio de dirección. Este resultado corrobora que incluso un retardo de tan solo 5 ms es suficiente para eliminar el ruido de contacto en los pulsadores utilizados.

Al igual que en la Actividad A, el mayor reto durante el montaje fue la identificación del pinout del display. La confusión entre ánodo y cátodo común se presentó tanto en la simulación de Proteus como en el circuito físico, ya que el símbolo del componente en el simulador no hace evidente visualmente cuál es la configuración del dispositivo. Una vez identificado y corregido el tipo de display — confirmando el cátodo común y ajustando los patrones del arreglo — los cuatro dígitos respondieron correctamente. Los resultados finales de las tres actividades fueron los esperados, con conteos estables, respuesta precisa al botón y visualización correcta en los displays.

Conclusiones Individuales

Ricardo Garza:

Esta práctica me dejó claro que buena parte del tiempo en el desarrollo de sistemas embebidos no se va en programar la lógica principal, sino en resolver los detalles de interfaz entre el software y el hardware real. El error de confundir ánodo y cátodo común en el display es un buen ejemplo: sabía perfectamente cómo funcionaba la multiplexación y tenía el arreglo `patron[]` correcto, pero el circuito no encendía porque la conexión física no correspondía al modelo que asumía el código. Eso me enseñó que antes de depurar

el programa hay que verificar que el modelo mental del hardware sea el correcto, revisando datasheet y pinout antes de simular.

La parte de ANSEL y ANSELH también me pareció muy valiosa. Ya habíamos aprendido esto en la práctica anterior con los botones, pero aquí se volvió a manifestar como un problema real: sin esa línea de inicialización, la interrupción externa de la Actividad B simplemente no respondía, y el error no era obvio porque el resto del código era correcto. Eso me refuerza el hábito de siempre inicializar explícitamente los registros de configuración antes de usar cualquier pin, sin dar nada por sentado en el estado de arranque del microcontrolador.

Eugenio Romo:

En esta práctica pude ver con claridad la diferencia entre dos enfoques para manejar eventos externos: la encuesta continua del botón dentro del loop principal, como se hace en la Actividad A, y la interrupción por hardware de la Actividad B. Aunque ambas logran un resultado similar para el usuario, la interrupción tiene la ventaja de que el programa no necesita dedicar tiempo de CPU a revisar el estado del pin en cada iteración, lo que en sistemas más complejos se traduce en mayor capacidad de respuesta y eficiencia.

El tema del rebote mecánico también fue algo que entendí mejor de manera práctica que teórica. Ver que una sola pulsación física generaba múltiples cambios de dirección en el contador fue suficiente para comprender por qué el debounce por software no es un paso opcional, sino una necesidad en cualquier sistema que lea botones físicos. El simple retardo de 5 ms resolvió completamente el problema, lo que muestra que soluciones sencillas bien aplicadas son suficientes para muchos casos reales.

Referencias:

[1] Merino, L. A. (2007). Microcontroladores PIC: Diseño práctico de aplicaciones (2.^a ed.). McGraw-Hill.

[2] Microchip Technology. (2009). PIC16F882/883/884/886/887 Data Sheet.

<https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>